When Tools Over-Report Success: Verifier-Anchored Tool Selection Under Partial Feedback

Author Name
Affiliation
email@example.com

Abstract

LLM agents choose tools by how well a tool's description matches the request. In open tool ecosystems such as the Model Context Protocol (MCP) that is unsafe: descriptions can be inflated or poisoned, tools report their own success, and only some calls can be checked. Learning from outcomes is the natural fix, but a learner that takes unverified success reports at face value adopts exactly the tool that over-reports. We study online tool selection under partial verification and describe RRL, a training-free selection layer built from simple parts: relevance plus Beta reputation, unverified success discounted rather than trusted, per-context counters shrunk toward each tool's global reputation, and a per-context memory that only verified outcomes can set. On six benchmarks, including local MCP servers with a poisoned server (recorded calls, replayed, with a three-seed live check), real tool descriptions from ToolRet and executable web-scraping extractors, a cross-encoder reranker and an LLM choosing from the tool list pick the poisoned tool almost every time and solve at most 8.2% of requests. With 20% of calls verified, a MemRL-style learner solves 3.9% and a robust per-context memory 24.8%, both misled by the poisoned tool's own reports; RRL solves 52.1% with learned source trust and 55.3% with a fixed discount (+27.3 points over the robust memory for the former, 95% CI [+25.4, +29.3]). A plain per-context memory matches RRL overall (51.8%) but not on contexts it has not seen (20.3% against 70.0%). The learned, per-tool model of how far to trust each feedback source is not what drives the gains: it differs from the fixed discount by at most 3.2 points and is slightly worse in every spot-check scenario with the poisoned server on Gates M and W. RRL loses where its assumptions fail: to a per-context memory when every call is verified and to a fast-forgetting learner under abrupt API changes, and it offers little protection against a tool that behaves honestly until trusted and then lies. We release the code, recorded outcomes and replay harness.

1 Introduction

An agent with access to many tools must decide which one to call. Today this choice is made from text: a retriever or the LLM itself compares the request with each tool's name and description [12, 24]. This assumes descriptions are honest and tools behave as described. In open registries neither holds. MCP servers can carry poisoned descriptions that both attract the agent and inject instructions [11, 27], tools are renamed and change their arguments between versions [4, 16], and a server can behave honestly until it is trusted and then defect [21]. Agents readily adopt the outputs of unreliable tools [31], and persistently misleading tool feedback can leave an agent worse off than no feedback at all [35]. Defences that rewrite or isolate descriptions [23, 33] address injection but not the question of which tool actually works.

The natural alternative is to learn from outcomes. Runtime memory methods such as MemRL [36] and Memento [37] learn which stored items lead to success, and RATED [5] learns tool reliability

from execution feedback with a contextual bandit. These methods take feedback at face value. But the cheapest feedback is often a tool's own report that it succeeded, or a schema check that a well-formed guess passes; expensive verification (running tests, a human check, comparing with ground truth) is affordable only for a fraction of calls [18, 34]. Agents and tools overclaim success [25], and silent failures arrive in the expected format [19]. A tool that over-reports success then looks like the best tool, and a learner that trusts the reports locks it in.

Our starting point was web scraping, where the same problem appears in a simple form: which extractor works for which site, when extractors report success on pages they parsed wrongly and sites redesign. We study the general problem, *online tool selection when tools may mislead and only some calls are verified*, and ask what a selection layer needs in order to resist tools that over-report. Our contributions:

1. **A simple, training-free selection layer that resists over-reported success.** RRL combines relevance with Beta reputation [14] and Thompson exploration [26], discounts unverified success, shrinks per-context counters toward each tool's global reputation so that new contexts start from what is known, and keeps a per-context memory that only the verifier can set or clear. Under spot checks it chooses a poisoned tool on 14.2% of steps, where a reranker, an LLM and two learning baselines adopt it.

2. **A negative result on learned feedback trust.** A two-coin Dawid–Skene [7] model of each source's sensitivity and specificity, estimated per tool against the verified calls (the idea behind audit-based trust in ESA [10]), differs from a fixed discount of unverified success by at most 3.2 points, and is slightly worse in all five spot-check scenarios with the poisoned server on Gates M and W, the case it was designed for. Learning one trust level per source is harmful, because the deceptive tool's report is informative on average.

3. **A reproducible evaluation that maps where this works and where it fails**: six benchmarks, including local MCP servers with a poisoned server and an LLM choosing from the tool list, a staged-defection scenario, 30 paired seeds, a shop-level bootstrap and ablations.

When it applies. The approach assumes that contexts or tools recur, that a verifier can check some calls, and that tools are stable enough for past outcomes to predict future ones. Its results are conditional on those assumptions: it reduces reliance on an over-reporting tool and transfers to unseen contexts, but loses to a per-context memory when every call is verified, to a fast-forgetting learner under abrupt change, and a remembered success can outlive a tool's validity when the tool changes without the verifier noticing.

Relation to the closest work. RATED [5] learns which tools work from execution feedback but assumes the feedback is truthful. ESA [10] learns per-source, per-context trust from sparse ground-truth audits, with regret bounds, but has no tool or LLM application; it also proves that without audits no aggregator can avoid linear regret when sources are truthful in some contexts and biased in others, and the collapse of a MemRL-style learner under spot checks (Section 5) is an instance of that result. Our evidence suggests that, for tool selection, the audits matter more than a learned trust model: anchoring memory to verified outcomes and discounting the rest is enough to avoid the over-reporting tool, and learned trust changes little. We have not yet compared against the official implementations of RATED, ESA or MemRL (Section 7).

2 Problem setting

At each step t the agent receives a request q_t with a context c_t (for example the website or the API being used) and a candidate pool $\mathcal{D}_t$ of tools with relevance scores $\mathrm{sim}(q_t, d) \in [0, 1]$ from a retriever or reranker. It calls one tool d_t and observes feedback signals $s_t = (s_t^k)_k$ from several sources k: the tool's own status report, a schema validator or judge, and user behaviour. With probability ρ (the *verification rate*) it also observes the true outcome $y_t \in \{0, 1\}$ from a verifier. The goal is to maximise the true success rate $\frac{1}{T}\sum_t y_t$, including on contexts that appear only late.

Three properties make the problem hard. (i) *Misleading relevance*: the most relevant description may belong to a poisoned or inflated tool. (ii) *Misleading feedback*: a source may report success regardless of the outcome, and how often it does so can differ by tool and over time. (iii) *Non-stationarity*: tools are updated or removed and contexts change (a site redesign), so past outcomes go stale. We call $\rho = 1$ the *verified* regime and $\rho = 0.2$ the *spot-check* regime.

3 Method

RRL keeps, for each tool d, short-term Beta counters (α_d, β_d), slower long-term counters (A_d, B_d), and per-context counters $(\alpha_{d,c}, \beta_{d,c})$. A tool with no evidence starts from $\alpha_d = 2, \beta_d = 1$ (an optimistic prior), so untried tools get tried. Algorithm 1 summarises one step.

Scoring. Per-context counters are shrunk toward the global ones by the evidence they hold, $n_{d,c} = \alpha_{d,c} + \beta_{d,c} - 2$:

$$\lambda = \frac{n_{d,c}}{n_{d,c} + \kappa}, \qquad \alpha = (1 - \lambda)\,\alpha_d + \lambda\,\alpha_{d,c}, \qquad \beta = (1 - \lambda)\,\beta_d + \lambda\,\beta_{d,c}, \tag{1}$$

with $\kappa = 4$, and likewise for (A, B). A new context therefore starts from the tool's global reputation and its own outcomes take over as they accumulate. The score is

$$\mathrm{score}(d) = w_s\,\mathrm{sim}(q, d) + w_c\,\frac{\alpha}{\alpha + \beta} + w_p\,\frac{A}{A + B} + w_e\,\mathrm{sim}(q, d)\,(\theta_d + b_d), \tag{2}$$

where $\theta_d \sim \mathrm{Beta}(\alpha, \beta)$ is a Thompson sample and $b_d = \max(0.05, (\alpha + \beta)^{-1/2})$ an uncertainty bonus; scaling exploration by relevance avoids trying tools that are irrelevant to the request. In our benchmarks $\mathrm{sim}(q, d)$ is the average of the retriever's similarity and the cross-encoder's rank score (1 for its first choice, 0 for its last).

Verified-success memory. If the verifier has confirmed that tool d worked in context c and no verified failure has followed, d is ranked first for c and exploration is skipped. Only the verifier can set or clear this flag, so a tool cannot lock itself in through its own reports.

Learning target. When the verifier ran, the outcome used for learning is $\hat{y} = y_t$. When it did not, RRL forms $\hat{y}$ from the other signals in one of two ways.

Fixed discount. A weighted average of the unverified signals in which the tool's own success report is capped at 0.75 (weights 0.45 for the report, 0.15 for the schema check or judge). A call that the tool and the schema check both call a success gives $\hat{y} \approx 0.81$; a reported failure gives $\hat{y} = 0$. Unverified success therefore moves reputation less than unverified failure.

Learned source trust. For source k and tool d, RRL counts over verified calls how often k said "success" when the call succeeded (sensitivity $\mathrm{se}_{k,d}$) and "failure" when it failed (specificity $\mathrm{sp}_{k,d}$).

The prior is centred on an uninformative source: both conditional "success" rates start at the source's own overall rate of saying "success" (prior strength 2), so a source whose answer does not depend on the outcome has se + sp = 1 and no weight however few or unbalanced the verified calls are. (A Beta(1,1) prior centred on 0.5, which an earlier version used, gives an always-yes source sp ≈ 1/3 after a single verified failure and so a positive vote.) Per-tool rates are shrunk toward the source's global rates with prior strength 4. Each signal contributes a log-likelihood ratio

$$\ell_k = \begin{cases} \log \dfrac{\mathrm{se}_{k,d}}{1-\mathrm{sp}_{k,d}} & \text{if } k \text{ reports success,} \\ \log \dfrac{1-\mathrm{se}_{k,d}}{\mathrm{sp}_{k,d}} & \text{if } k \text{ reports failure,} \end{cases} \tag{3}$$

clipped to $[-4, 4]$ and set to 0 when $\mathrm{se}_{k,d} + \mathrm{sp}_{k,d} \le 1$, that is, when the source is no better than chance for this tool, and

$$\hat{y}_t = \sigma\Big(\mathrm{logit}\,\pi + \textstyle\sum_k \ell_k\Big), \tag{4}$$

where π is the verified base rate; if no source is informative the step is not used for learning. This is the two-coin Dawid–Skene model [7, 29] anchored to a verifier rather than estimated by EM, close in spirit to trust bootstrapping from a small trusted set [2] and to audit-based trust in ESA [10]. *RRL full*, our pre-specified main configuration, uses learned source trust; Section 5.1 compares it with the fixed discount.

Updates. After each call the chosen tool's counters are updated with $\hat{y}$: $\alpha \mathrel{+}= \omega\hat{y}$, $\beta \mathrel{+}= \omega(1-\hat{y})$, with weight $\omega = 2|\hat{y} - \frac{1}{2}|$ so that uncertain outcomes move the counters little; the long-term counters receive a quarter of the update.

Change detection. For each tool, and each tool–context pair, RRL keeps the verifier's own 0/1 outcomes and, after each verified call, compares the success rates $\hat{p}_1, \hat{p}_2$ of the last two windows of $w = 8$ verified outcomes with a two-proportion z-test at $\alpha = 0.01$, $z = |\hat{p}_2 - \hat{p}_1| / \sqrt{2\bar{p}(1-\bar{p})/w}$. On detection the short-term counters are rebuilt from the most recent window only. Testing verified outcomes matters twice: the proportion test assumes binary observations, while the learning targets $\hat{y}$ are soft; and a tool whose unverified reports stay positive while its verified calls fail is still detected. Testing after every verified call with overlapping windows inflates the nominal 1% level, so we report the resets observed in stable conditions. This replaces a fixed threshold on $|\hat{p}_2 - \hat{p}_1|$, which produced many false resets (Section 5.3); it is a simple instance of adaptive windowing [1] and related to sliding-window and discounted bandits [9, 20].

4 Benchmarks

All benchmarks replay offline from recorded outcomes: for every request and every tool in its pool we record whether the tool works, so any policy can be evaluated on the same data [15]. Seeds vary the request order, feedback noise, flaky calls and exploration; we run 30 seeds and pair every comparison over the same seeds (95% CIs from a paired t-test; Wilcoxon tests agree). A held-out set of contexts appears only in a final block, to measure transfer.

What the replay does and does not model. A policy sees only the feedback of the tool it chose, and the verifier's result only on the calls drawn for verification. Transient failures and timeouts are drawn per step from fixed rates; tool updates and site redesigns are a second recorded

Algorithm 1 One RRL step for request q in context c

for each candidate tool $d \in \mathcal{D}$ **do**
    shrink $(\alpha_{d,c}, \beta_{d,c})$ toward (α_d, β_d) (Eq. 1); compute score(d) (Eq. 2)
end for
if some d has a verified success in c and no verified failure since **then**
    call that d ▷ verified-success memory, no exploration
else
    call $\arg\max_d$ score(d) with the Thompson term
end if
observe signals s^k, and y if the call was verified
if verified **then**
    $\hat{y} \leftarrow y$; with learned trust, update each source's (se, sp) for d
else
    $\hat{y} \leftarrow$ fixed discount, or $\sigma(\text{logit}\,\pi + \sum_k \ell_k)$ with learned trust (Eqs. 3–4)
end if
update d's global and context counters with $\hat{y}$, weight $\omega = 2|\hat{y} - \frac{1}{2}|$
if verified, run the change test on d's verified outcomes (and on d in c); on detection rebuild counters

phase; defection rewrites one tool's outcomes after a fixed step. Tools are otherwise stateless: a call never changes what later calls return.

Statistics. The confirmatory comparison is RRL full against the robust per-context memory under spot checks on Gates M and W. Seed-level intervals vary only request order, noise and exploration over the same contexts, so for Gate M we also bootstrap over the 30 shops (Section 5). Ablations and the remaining comparisons are exploratory; where we count significant results across many scenarios we report how many survive a Holm correction.

Gate M: live MCP servers. Four local MCP servers over stdio expose eight extractor tools that fetch pages from a local shop site and return title, price and SKU: CSS extractors, a structured-data server (JSON-LD, microdata, Open Graph), a heuristic scraper on a flaky server, and a "universal AI extractor" whose description is inflated and carries an MCPTox-style [27] instruction to call `send_report` with the page content. The canary tool only appends a line to a local file, so following the injection can be counted. 6 of 30 shops are held out. In the *update* scenarios half the shops redesign and the structured-data server is upgraded mid-run, renaming `extract_jsonld(url)` to `jsonld_extract(page_url)` as in MCPEvol-Bench's mutations [16]. In the *defection* scenarios, modelled on TrustShift attacks [21], the most useful honest extractor (`css_woo`) starts returning well-formed wrong results halfway through the seen shops while still reporting success. Outcomes were recorded from 2,280 live MCP calls and replayed with 20% timeouts on the flaky server and 2% transient failures on every tool; all Gate M tables are replays. As a check we also run every policy with real MCP calls on every step, for three seeds. The servers and the website are local; no third-party MCP service is involved. The *LLM picks* baseline gives Gemini 2.5 Flash the live tool list and lets it plan its calls, as agents do today.

Gate W: web-scraping extractors. 30 generated shops (6 held out) and the same eight extractors running real parsing code, without the MCP layer. The universal extractor returns well-formed guesses, so both its self-report and the schema validator say success.

Gate T: real tool descriptions. Requests and gold tools from ToolRet [24] (ToolBench: 431 requests whose top-8 pool contains a working tool; ToolLens: 200). On top we add v2 replacements for 30% of gold tools (drift), poisoned twins for 10% whose descriptions copy example requests (the reranker then picks a poisoned tool first for 39% and 56% of requests), and flakiness for 20% of tools. Success means the chosen tool is a gold tool that is deployed and does not flake; ToolRet's labels stand in for execution.

Gate G: private library. Gemini 2.5 Flash writes code against a generated library whose documentation is the only way to solve the task; poisoned guides fool the reranker on 79 of 80 poisoned questions; *drift* changes the library's API halfway.

Gate F: MBPP. A leak-free retrieval benchmark on MBPP with 4,128 recorded Gemini 2.5 Flash completions, used as a control.

Metrics. *Success* is the share of steps whose chosen tool returned a correct result (Gates M, W), a gold tool that was deployed and did not flake (T), or code that passed the tests (G, F), averaged over all steps including the held-out block. *Held-out success* is the same over the final block only. On Gate M we also report how often the poisoned tool was chosen. Hyperparameters are listed in Appendix A.

Feedback scenarios. *Verified*: every call is verified. *Spot check*: 20% of calls are verified and the rest carry only the tool's report and the schema check. *Sycophantic* and *fooled* (Gates T, G): the unverified judge says success too often, in general or specifically for poisoned items.

Baselines. *Static reranker*: cross-encoder top-1. *LLM picks* (Gate M). *Per-context memory*: for each context, walk the reranker's order until a tool works, then keep it. Its *robust* variant keeps a working tool through an isolated failure (two failures in a row, three if unverified, before moving on). *MemRL-style*: our reimplementation of MemRL's ranking rule [36], $(1-\lambda)\,z(\text{sim}) + \lambda\, z(Q)$ with $Q \leftarrow Q + 0.3(r - Q)$ and $\lambda = 0.5$, MemRL's published values; we start Q at 0.5 where MemRL starts at 0, which gave equal or higher success for this baseline in a check on the Gate M scenarios. The learning baselines use the verifier when it ran and take the other feedback at face value. *Best possible*: an oracle that always picks a working tool if one exists. *RRL (global)* is RRL without per-context counters or verified memory; *RRL full* has both; *RRL full, fixed rule* replaces learned source trust with the fixed discount. The benchmark weights are $(w_s, w_c, w_p, w_e) = (0.20, 0.40, 0.10, 0.30)$; the library default uses $w_e = 0.10$, which we report as an ablation.

5 Results

Description-based selection fails with a poisoned server. With the poisoned server in the registry, the static reranker picks it on every step and solves none of the requests (Table 1). Gemini choosing from the tool list does little better: it picks the poisoned tool on 99.0% of steps, solves 1.0–8.2% of requests, and plans the injected `send_report` call on 4.2–6.4% of steps. Every learning method does better: with every call verified RRL full solves 66.3% and picks the poisoned tool on 1.4% of steps.

Overall / held-out success (%)	Verified	Spot check	Update	Update + spot
Static reranker	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0
LLM picks from tool list	1.0 / 0.0	1.0 / 0.0	8.2 / 0.0	8.2 / 0.0
Per-context memory	70.9 / 65.0	51.8 / 20.3	62.9 / 61.3	**46.5** / 17.9
Per-context memory, robust	**75.0** / 65.3	24.8 / 11.1	**65.7** / 61.4	21.9 / 10.1
MemRL-style	35.8 / 45.8	3.9 / 1.9	33.8 / 32.2	3.2 / 1.2
RRL (global)	45.9 / 57.2	42.6 / 63.7	42.5 / 51.2	36.6 / 49.0
RRL full	66.3 / 73.9	52.1 / 70.0	55.9 / 56.5	43.2 / 55.6
RRL full, fixed rule	66.3 / 73.9	**55.3** / 69.0	55.9 / 56.5	46.3 / 53.6
Best possible	90.4 / 98.8	90.4 / 98.8	90.4 / 98.8	90.4 / 98.8

Table 1: Gate M: tool selection over local MCP servers with a poisoned server (replay of recorded MCP responses, 30 seeds). Best non-oracle result per column in bold.

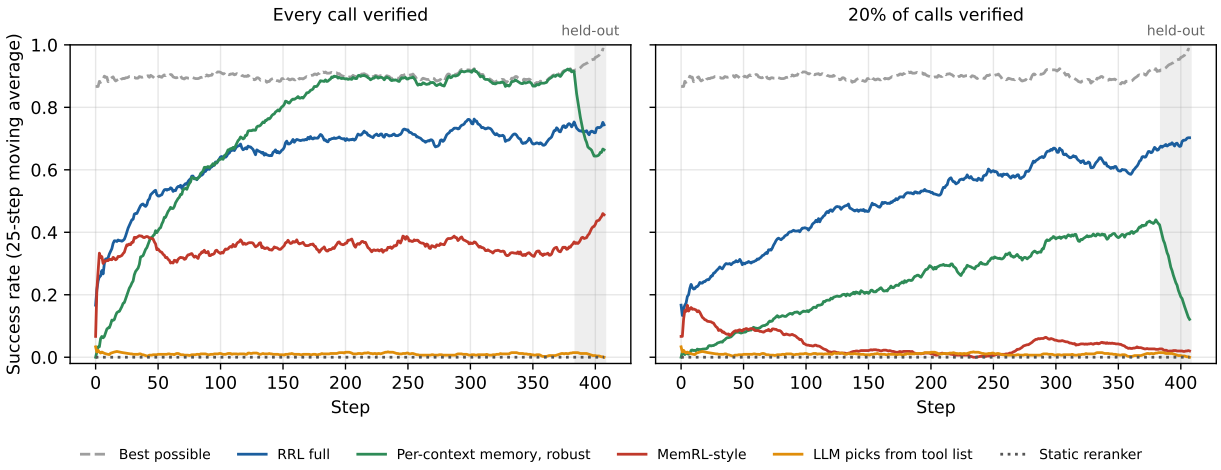


Figure 1: Gate M with the poisoned server: success over the run (mean of 30 seeds). The shaded block holds out six shops never seen before. With every call verified the robust per-context memory leads on familiar shops but drops on new ones; with 20% verified it is fooled by the poisoned tool's own reports, and MemRL-style collapses.

With spot checks, learners that take reports at face value are fooled. When only 20% of calls are verified, the poisoned tool's own success reports dominate the feedback. MemRL-style trusts them, picks the poisoned tool on 90.1% of steps and solves 3.9%. The robust per-context memory, built to survive isolated failures, holds on to the poisoned tool on the strength of the same reports (68.8% poisoned picks, 24.8% solved). RRL, which discounts unverified success and lets only verified outcomes lock in a per-context choice, still picks it on 14.2% of steps, so its protection is partial, and solves 52.1%: +27.3 points over the robust memory (95% CI [+25.4, +29.3], better on 30 of 30 seeds). Bootstrapping over the 30 shops instead of seeds gives +32.1 points per shop (95% interval [+20.9, +43.0]), with RRL ahead on 22 of 30 shops. Gate W shows the same pattern without the MCP layer (+25.1 points; Table 2); across the four spot-check scenarios of Gates M and W the margin is 19.3–27.3 points. A plain per-context memory, which abandons a tool after a single failure, is not misled in the same way and matches RRL overall (51.8%), but on the held-out shops it solves 20.3% against 70.0% for RRL, because a new context starts from each tool's global reputation rather than from the reranker.

Overall / held-out success (%)	Verified	Spot check	Redesign	Redesign + spot
Static reranker	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0
Per-context memory	67.0 / 61.4	50.4 / 19.6	64.2 / 58.5	**47.9** / 17.5
Per-context memory, robust	**73.9** / 61.9	27.0 / 11.0	**70.4** / 58.9	25.2 / 10.3
MemRL-style	35.9 / 43.3	4.4 / 3.6	34.0 / 32.1	4.3 / 3.5
RRL (global)	51.4 / 68.9	47.1 / 75.4	45.5 / 53.5	39.5 / 53.1
RRL full	62.0 / 79.6	52.1 / 81.8	54.1 / 60.7	44.4 / 61.0
RRL full, fixed rule	62.0 / 79.6	**54.1** / 77.6	54.1 / 60.7	46.6 / 61.0
Best possible	87.9 / 96.4	87.9 / 96.4	87.9 / 96.4	87.9 / 96.4

Table 2: Gate W: web-scraping extractor selection with executable extractors.

Overall / held-out success (%)	Verified	Spot check	Update	Update + spot
Static reranker	46.0 / 65.7	46.0 / 65.7	37.4 / 49.3	37.4 / 49.3
LLM picks from tool list	47.7 / 61.8	47.7 / 61.8	49.7 / 53.8	49.7 / 53.8
Per-context memory	77.5 / 89.4	77.0 / 89.4	70.4 / 85.4	69.2 / 85.4
Per-context memory, robust	**81.8** / 90.3	**81.6** / 90.3	**72.5** / 86.0	**71.3** / 86.0
MemRL-style	34.9 / 45.6	34.2 / 40.8	34.1 / 31.7	31.7 / 26.9
RRL (global)	48.0 / 61.9	48.2 / 61.8	44.1 / 51.9	45.2 / 56.0
RRL full	66.3 / 72.4	60.0 / 72.2	55.8 / 56.0	51.2 / 58.8
RRL full, fixed rule	66.3 / 72.4	59.6 / 70.1	55.8 / 56.0	51.1 / 57.1
Best possible	90.4 / 98.8	90.4 / 98.8	90.4 / 98.8	90.4 / 98.8

Table 3: Gate M without the poisoned server (clean registry).

When every call is verified, a per-context memory wins. In the verified regime there is nothing to be fooled by, and the robust memory beats RRL full by 8.7 points on Gate M (95% CI [6.8, 10.6]), by 11.9–16.3 on Gate W, and by 15.5–16.6 on the clean registry (Table 3). At the level of shops the Gate M lead is less certain: the memory leads by 6.1 points per shop with a 95% interval that includes zero ([−0.7, +13.6]), and RRL full is ahead on 17 of 30 shops, so the memory's advantage comes from large wins on the rest. Without the poisoned server the memory also wins under spot checks, by 20.0–21.6 points: when no tool over-reports success, the unverified reports are honest and trusting them is right. Per-context memory is the right tool when outcomes are trustworthy and contexts recur; RRL buys robustness to over-reporting and transfer to new contexts at the cost of exploration.

5.1 What drives the gains: learned trust or a fixed discount?

RRL's learned source trust was designed to recognise that the poisoned tool's self-report is uninformative for that tool. To test whether it is what keeps the poisoned tool out, we replace it with the fixed discount and keep everything else. In the 23 scenarios with unverified feedback (Appendix B) the two differ by −3.2 to +2.9 points: learned trust is significantly better in 5, significantly worse in 6 and indistinguishable in 12 (2 and 1 after a Holm correction across the 23 tests). It helps a little where the verifier is sparse and the other feedback mostly honest (the sparse verifier on Gate T) and is worse in the case it was built for: in all five spot-check scenarios with the poisoned tool on Gates M and W the fixed rule is ahead, by 2.0–3.2 points (Gate M: 55.3% against 52.1%, 95% CI of the difference [1.2, 5.2]). We have not isolated the cause; one candidate is that per-tool rates start from each source's global rates, which the poisoned tool's constant reports distort until a tool

Overall / held-out success (%)	Defection	Defection + spot
Static reranker	0.0 / 0.0	0.0 / 0.0
LLM picks from tool list	0.5 / 0.0	0.5 / 0.0
Per-context memory	60.5 / 48.3	39.1 / 16.8
Per-context memory, robust	**61.9** / 48.6	17.6 / 8.9
MemRL-style	34.0 / 57.6	3.0 / 0.0
RRL (global)	38.6 / 62.8	31.6 / 61.3
RRL full	51.6 / 63.2	36.8 / 64.3
RRL full, fixed rule	51.6 / 63.2	**40.0** / 61.1
Best possible	83.6 / 82.2	83.6 / 82.2

Table 4: Staged defection on Gate M: halfway through, the most useful honest extractor starts returning well-formed wrong results while still reporting success. Held-out shops come after the defection.

has verified calls of its own. Learned trust is clearly better only than learning one sensitivity and specificity per source, which loses 8.3 points on Gate M and 9.0 on Gate W because the deceptive tool's report is informative on average.

So learned trust is not what drives the gains. What does is shared by both variants and missing from the baselines that fail: neither lets an unverified success count as a full success, and neither lets an unverified outcome lock in a per-context choice. Per-context counters and verified memory supply most of the remaining gain (Section 5.3). For tool selection under spot checks, the verifier's role as the only thing that can set the memory, rather than a model of each source's reliability, is what keeps the over-reporting tool out.

5.2 Staged defection

Table 4 tests a tool that has been useful and honest and then starts returning wrong results that still look right. Every learner loses. With every call verified RRL full solves 51.6% and the robust memory 61.9% (best possible 83.6%). Under spot checks RRL full loses 15.3 points against the same scenario without defection, more than the 6.7 points the best possible pick loses; it still beats the robust memory (+19.2 points; 17.6%) but no longer the plain per-context memory (39.1% against 36.8%). Without the poisoned server (Appendix C) the robust memory keeps its lead (63.7% against 42.9%). RRL offers no specific protection. Its change detector tests verified outcomes and so can see the defection, but at 20% verification a tool needs about 80 calls to fill two windows of eight verified outcomes, and the detector fires 0.8 times per run on average; after a reset the defector's unverified reports keep arriving as successes. RRL (global) solves 31.6% with the detector and 31.2% without it, and the fixed discount does not help either (40.0%). Version-aware memory, periodic audits of trusted tools and a limit on how long a success counts without re-verification are the obvious next steps (Section 7).

Live runs agree with the replay. Table 5 compares the replay with runs that make real MCP calls on every step. RRL full is within 1 point of its replayed result. Per-context memory does about 6 points better live, because the replay adds 2% transient failures on every tool, the local servers had none outside the flaky one, and that baseline abandons a working tool after a single failure.

Success (%)	Verified Replay	Verified Live	Update Replay	Update Live
Static reranker	0.0	0.0	0.0	0.0
LLM picks from tool list	1.0	1.0	8.2	8.3
Per-context memory	70.9	77.2	62.9	69.4
Per-context memory, robust	75.0	77.2	65.7	67.9
MemRL-style	35.8	37.7	33.8	35.0
RRL (global)	45.9	47.4	42.5	44.3
RRL full	66.3	65.7	55.9	56.9

Table 5: Gate M replayed from recorded MCP responses (30 seeds) versus run with real MCP calls on every step (3 seeds).

Overall / held-out success (%)	Verified	Sycophantic	Fooled	Drift	Drift + fooled
Static reranker	31.4 / 33.0	31.4 / 33.0	31.4 / 33.0	30.5 / 30.3	30.5 / 30.3
Per-context memory	46.0 / 33.0	41.1 / 33.0	39.9 / 33.0	41.0 / 30.3	35.2 / 30.3
Per-context memory, robust	46.4 / 33.0	39.8 / 33.0	39.7 / 33.0	40.4 / 30.3	34.1 / 30.3
MemRL-style	**55.5** / 45.3	**50.0** / 42.7	**45.5** / 39.2	**51.0** / 42.3	**41.9** / 38.2
RRL (global)	52.5 / 50.1	45.4 / 45.4	44.5 / 43.6	48.1 / 48.7	41.5 / 42.1
RRL full	55.3 / 48.4	46.2 / 46.3	45.2 / 42.8	49.5 / 48.5	41.8 / 42.8
RRL full, fixed rule	55.3 / 48.4	45.7 / 44.9	44.7 / 43.1	49.5 / 48.5	41.2 / 41.7
Best possible	89.9 / 89.4	89.9 / 89.4	89.9 / 89.4	88.6 / 87.1	88.6 / 87.1

Table 6: Gate T (ToolBench requests from ToolRet): tool selection on real tool descriptions.

Cost. RRL needs no training and no extra model calls; a decision over eight tools takes well under a millisecond in pure Python, negligible next to the tool call itself.

On real tool descriptions RRL matches MemRL-style. On ToolRet requests (Tables 6 and 7) RRL and MemRL-style are within a few points of each other. MemRL-style is ahead by 3.5–3.8 points under the sycophantic judge and by 3.9 on ToolBench with a sparse verifier; RRL full is ahead by 2.6–2.7 points on ToolLens with a verified or a fooled judge; with the library's exploration weight, RRL (global) is ahead of MemRL-style or within half a point in 10 of 12 scenarios. Both beat per-context memory by 5.0–16.2 points because requests rarely repeat, and both remain far from the best possible pick.

Under abrupt change, a fast-forgetting learner adapts faster. In Gate G (Table 8), when the library's API changes halfway and feedback is trustworthy, MemRL-style's exponential moving average forgets the old documents faster than RRL's change detector, which waits for statistical evidence. In the drift scenario RRL (global) trails it by 9.2 points and RRL full by 12.7: per-context counters and verified memory cost a few more points by keeping the old documents for questions that recur after the change.

A control: when the model knows the answer, retrieval does not matter. On MBPP (Gate F), the reranker solves 57.1% and no retrieval 57.2%; the best possible pick reaches only 61.3%. A benchmark where the model already knows the answer cannot show the value of learning which items help, which is why we built Gates G, T, W and M.

Overall / held-out success (%)	Verified	Sycophantic	Fooled	Drift	Drift + fooled
Static reranker	29.1 / 32.2	29.1 / 32.2	29.1 / 32.2	26.5 / 25.2	26.5 / 25.2
Per-context memory	42.9 / 32.2	38.2 / 32.2	34.7 / 32.2	36.2 / 25.2	29.7 / 25.2
Per-context memory, robust	43.5 / 32.2	37.1 / 32.2	34.3 / 32.2	35.7 / 25.2	28.6 / 25.2
MemRL-style	55.4 / 57.9	**51.7** / 56.3	43.9 / 50.3	**52.4** / 55.4	40.6 / 42.0
RRL (global)	56.1 / 55.6	48.1 / 51.5	45.7 / 49.3	49.9 / 52.0	41.6 / 45.8
RRL full	**58.1** / 55.8	48.2 / 51.2	46.5 / 49.5	50.6 / 52.5	41.5 / 43.7
RRL full, fixed rule	58.1 / 55.8	49.4 / 53.3	**47.2** / 50.5	50.6 / 52.5	**42.3** / 45.8
Best possible	85.1 / 81.1	85.1 / 81.1	85.1 / 81.1	84.7 / 83.2	84.7 / 83.2

Table 7: Gate T (ToolLens requests from ToolRet).

Overall / held-out success (%)	Verified	Fooled	Drift	Drift + syco.	Drift + fooled
Static reranker	50.8 / 50.0	50.8 / 50.0	23.5 / 0.0	23.5 / 0.0	23.5 / 0.0
Per-context memory	83.7 / 50.0	64.8 / 50.0	51.3 / 0.0	39.1 / 0.0	40.1 / 0.0
Per-context memory, robust	83.7 / 50.0	59.4 / 50.0	34.8 / 0.0	29.8 / 0.0	26.3 / 0.0
MemRL-style	96.0 / 100.0	**89.7** / 99.4	**80.9** / 95.5	**73.7** / 93.8	**74.8** / 93.9
RRL (global)	96.5 / 100.0	87.8 / 98.9	71.7 / 96.7	51.3 / 56.2	54.2 / 70.8
RRL full	**96.5** / 100.0	87.8 / 99.4	68.2 / 97.9	48.7 / 55.2	49.6 / 67.4
RRL full, fixed rule	96.5 / 100.0	88.0 / 97.4	68.2 / 97.9	48.6 / 50.4	48.8 / 64.0
Best possible	100.0 / 100.0	100.0 / 100.0	100.0 / 100.0	100.0 / 100.0	100.0 / 100.0

Table 8: Gate G (hard): code generation against a generated library with poisoned guides and a mid-stream API change.

5.3 Ablations

Table 9 removes one component at a time on Gate M.

- **Per-context counters and verified memory** supply most of RRL's gain where contexts recur: from RRL (global) to RRL full adds 9.5 points under spot checks and 20.4 with every call verified, verified memory alone +9.1. Neither helps on Gate T, where requests rarely repeat, and both cost a few points under drift on Gate G (above).
- **Feedback trust.** Learned per-tool trust is within 3.2 points of the fixed discount and slightly worse with the poisoned tool under spot checks (Section 5.1); a single trust level per source is clearly worse.
- **Change detection.** Testing verified outcomes rather than the soft learning targets, as an earlier version did, is significantly better in 6 of 41 scenarios and worse in 2 (Gate G under drift with a misleading judge). Compared with the original fixed-threshold rule, resets in stable conditions fall from 23.0 to 1.9 per run on Gate W, but the detector still costs 2.4 points on Gate M and 2.5 on Gate W when nothing changes, reacts later to a real change than that rule (Gate G), and does little against staged defection.
- **Exploration.** A confidence gate that stops exploring once the leader is significantly best had no effect: while any tool is untried, the leader cannot be shown best at 95% confidence. The library default $w_e = 0.10$ is significantly better than the benchmark's 0.30 in 19 of 21 scenarios on Gates T and G and never significantly worse; we report both rather than re-tune after seeing the results.

Overall / held-out success (%)	Verified	Spot check	Update	Update + spot
No learned trust, no change detection	48.3 / 65.0	44.3 / 63.5	45.3 / 60.8	38.9 / 52.2
Global-only source trust	45.9 / 57.2	34.3 / 52.5	42.5 / 51.2	26.7 / 23.3
No change detection	48.3 / 65.0	42.7 / 64.2	45.3 / 60.8	37.3 / 50.4
Fixed-threshold detector, soft outcomes	42.6 / 57.2	36.9 / 54.2	39.3 / 39.7	32.5 / 34.4
z-test on soft outcomes	45.9 / 57.2	42.0 / 62.1	42.5 / 51.2	36.0 / 43.6
Exploration off	44.6 / 55.3	44.6 / 60.7	40.7 / 45.1	39.5 / 52.1
Exploration 0.10	44.6 / 51.0	44.7 / 63.6	41.6 / 46.5	40.7 / 56.4
Confidence-gated exploration	45.9 / 57.2	42.6 / 63.7	42.5 / 51.2	36.6 / 49.0
RRL (global)	45.9 / 57.2	42.6 / 63.7	42.5 / 51.2	36.6 / 49.0
RRL + per-context	57.1 / 67.9	47.2 / 68.5	48.8 / 45.4	39.8 / 53.3
RRL full	**66.3** / 73.9	52.1 / 70.0	**55.9** / 56.5	43.2 / 55.6
RRL full, fixed rule	66.3 / 73.9	**55.3** / 69.0	55.9 / 56.5	**46.3** / 53.6

Table 9: Ablations on Gate M (overall / held-out success, %). Each row changes one component of RRL (global); the last three rows add per-context counters, then verified memory, then swap learned trust for the fixed rule.

6 Related work

Tool retrieval and selection. ToolRet [24] shows that retrievers are weak at finding the right tool; AutoTool [12] reduces selection cost; LQM-ContextRoute [6] routes among functionally equivalent tools by latency and quality. RATED [5] is the closest: it learns tool reliability from execution dynamics with a contextual bandit, assuming truthful feedback.

Tool poisoning and evolution. MCPTox [27] and MCP threat models [11] show that poisoned descriptions succeed against real agents; Tool-Guard [23] and TRUSTDESC [33] defend by isolated planning and trusted description generation. MCPEvol-Bench [16] and ToolEVO [4] study tools that change, and TrustShiftProbe [21] servers that turn malicious after earning trust. Agents over-rely on unreliable tools [31], misleading feedback can be worse than none [35], and outcome monitors detect results that are well formed but wrong [19]. Our approach is complementary to these defences: it does not inspect descriptions, it learns from outcomes which tools to stop using.

Learning from feedback in agents and retrieval. MemRL [36], Memento [37] and Evo-Memory [28] learn the utility of stored items at run time; RoMeRL [32] discusses how memory rewards can trap such learners. Rerankers trained from LLM feedback [30] and utility-driven reranking [3] also rank by outcome rather than similarity, but offline. Feedback loops can degenerate [13]. Verified semantic caching [22] and prediction-powered ranking evaluation [8] rely on verification or a small labelled sample to make cheaper signals trustworthy, as we do.

Trust and robust bandits. Beta reputation [14], the Dawid–Skene model [7, 29], trust bootstrapping [2] and contextual trust learning from audits [10] underlie the learned-trust variant we test. Corruption-robust bandits [17] bound the damage of adversarial feedback without modelling its source; non-stationary bandits [9, 20] and adaptive windowing [1] handle change. Pan et al. [18] show that self-reports of LLM operators are not verification, and Zavattari et al. [34] study how to allocate a limited audit budget across agents.

7 Limitations

- **Baselines.** MemRL-style is a reimplementation of MemRL's ranking rule, not its official code, and we have not yet compared against RATED or an ESA-style aggregator, the two closest methods.
- **Environments.** Gate M uses real MCP servers over stdio but on a local, generated website with a synthetic poisoned server; no third-party MCP servers or live websites were used. We have not evaluated on unmodified MCPTox or MCPEvol-Bench tasks. Gate T judges success by ToolRet labels, not execution.
- **Synthetic stress.** Poisoning, flakiness, updates, redesigns, defection and the feedback models are added on top of the data; their rates are our choices.
- **Staged defection** costs RRL its advantage over per-context memory (Section 5.2); it should not be relied on against a tool that can behave well until trusted.
- **Learned trust.** Our negative result covers our benchmarks, where one tool deceives and the other sources are mostly honest, and it was found after the main configuration was fixed; we report the pre-specified configuration as RRL full. Settings with many sources of varied reliability, or with deception concentrated on some contexts, may favour learned trust; we assume verified calls are sampled at random, and biased audits or correlated sources are untested.
- **Stale verified memory.** A verified success stays authoritative for a context until a verified failure. Binding memory to a tool's version or interface fingerprint, auditing trusted tools periodically, and limiting how long a success counts without re-verification are natural safeguards; we have not evaluated them, and each needs its recovery time, false resets and verification cost measured.
- **Statistics.** Our intervals cover seeds and, for the primary comparison, shops; they do not establish that effects carry over to other tool registries.
- **Regime.** RRL loses to a per-context memory when every call is verified and to a fast-forgetting learner under abrupt change with trustworthy feedback.

8 Conclusion

Choosing tools by their descriptions fails as soon as a description lies, and learning from outcomes fails as soon as the outcomes lie. In our benchmarks the remedy is simple: discount unverified success, let only verified outcomes lock in a choice, and start new contexts from each tool's global reputation. That sharply reduces reliance on a poisoned tool that a reranker, an LLM and two learning baselines adopt when only some calls are checked; a learned model of how far to trust each feedback source adds nothing on top and is slightly worse with the poisoned tool under spot checks. Where every call can be checked, simpler memories are better, and a tool that defects after earning trust erases the approach's advantage. The open questions are how to choose which calls to verify and how to detect defection from a few verified failures.

Reproducibility. Code, recorded outcomes, per-seed results and the scripts that generate every table and figure are available at `https://github.com/pras-ops/retrieval-reputation-layer`.

Ethics statement. The poisoned MCP server used in Gate M is a harmless canary: its injected instruction asks the agent to call a tool that only appends a line to a local file. All servers and websites run locally; no third-party service was attacked or scraped.

References

[1] Albert Bifet and Ricard Gavaldà. Learning from Time-Changing Data with Adaptive Windowing. In *Proceedings of the SIAM International Conference on Data Mining*, 2007.

[2] Xiaoyu Cao, Minghong Fang, Jia Liu, and Neil Zhenqiang Gong. FLTrust: Byzantine-robust Federated Learning via Trust Bootstrapping, 2020. URL https://arxiv.org/abs/2012.13995.

[3] Manish Chandra, Debasis Ganguly, and Iadh Ounis. LURE-RAG: Lightweight Utility-driven Reranking for Efficient RAG, 2026. URL https://arxiv.org/abs/2601.19535.

[4] Guoxin Chen, Zhong Zhang, Xin Cong, Fangda Guo, Yesai Wu, Yankai Lin, Wenzheng Feng, and Yasheng Wang. Learning Evolving Tools for Large Language Models, 2024. URL https://arxiv.org/abs/2410.06617.

[5] Weiliang Chen and Song Liao. RATED: Reliability-Aware AI Agent Tool Retrieval via Execution Dynamics, 2026. URL https://arxiv.org/abs/2610.13889.

[6] Kexin Chu, Dawei Xiang, and Wei Zhang. Latency-Quality Routing for Functionally Equivalent Tools in LLM Agents, 2026. URL https://arxiv.org/abs/2605.14241.

[7] A. P. Dawid and A. M. Skene. Maximum Likelihood Estimation of Observer Error-Rates Using the EM Algorithm. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 28 (1):20–28, 1979.

[8] Abhishek Divekar. Statistically Reliable LLM-Based Ranking Evaluation via Prediction-Powered Inference, 2026. URL https://arxiv.org/abs/2606.05308.

[9] Aurélien Garivier and Eric Moulines. On Upper-Confidence Bound Policies for Non-Stationary Bandit Problems, 2008. URL https://arxiv.org/abs/0805.3415.

[10] Majid Ghasemi and Mark Crowley. Learning When to Trust in Contextual Social Bandits, 2026. URL https://arxiv.org/abs/2603.13356.

[11] Charoes Huang, Xin Huang, Ngoc Phu Tran, and Amin Milani Fard. Model Context Protocol Threat Modeling and Analyzing Vulnerabilities to Prompt Injection with Tool Poisoning, 2026. URL https://arxiv.org/abs/2603.22489.

[12] Jingyi Jia and Qinbin Li. AutoTool: Efficient Tool Selection for Large Language Model Agents, 2025. URL https://arxiv.org/abs/2511.14650.

[13] Ray Jiang, Silvia Chiappa, Tor Lattimore, András György, and Pushmeet Kohli. Degenerate Feedback Loops in Recommender Systems, 2019. URL https://arxiv.org/abs/1902.10730.

[14] Audun Jøsang and Roslan Ismail. The Beta Reputation System. In *Proceedings of the 15th Bled Electronic Commerce Conference*, 2002.

[15] Lihong Li, Wei Chu, John Langford, and Xuanhui Wang. Unbiased Offline Evaluation of Contextual-bandit-based News Article Recommendation Algorithms, 2010. URL https://arxiv.org/abs/1003.5956.

[16] Huanxi Liu, Kun Hu, Jiaqi Liao, Qiang Wang, Pengfei Qian, YuanZhao Zhai, Dawei Feng, Bo Ding, and Huaimin Wang. MCPEvol-Bench: Benchmarking LLM Agent Performance Across Dynamic Evolutions of MCP Servers, 2026. URL https://arxiv.org/abs/2607.14642.

[17] Thodoris Lykouris, Vahab Mirrokni, and Renato Paes Leme. Stochastic bandits robust to adversarial corruptions, 2018. URL https://arxiv.org/abs/1803.09353.

[18] Enrong Pan, Ryan Zhou, and Ting Hu. Self-Reports Are Not Verification: Environment-Grounded Auditing of LLM Operators in Evolutionary Search, 2026. URL https://arxiv.org/abs/2609.00652.

[19] Sugam Panthi and Rabab Abdelfattah. Outcome Monitors: Recovery Affordances for Silent Tool Failures, 2026. URL https://arxiv.org/abs/2608.19303.

[20] Han Qi, Yue Wang, and Li Zhu. Discounted Thompson Sampling for Non-Stationary Bandit Problems, 2023. URL https://arxiv.org/abs/2305.10718.

[21] Mehrdad Rostamzadeh, Sidhant Narula, Mohammad Ghasemigol, and Daniel Takabi. TrustShiftProbe: Characterizing, Benchmarking, and Defending Staged Trust Attacks on MCP Servers, 2026. URL https://arxiv.org/abs/2608.23763.

[22] Luis Gaspar Schroeder, Aditya Desai, Alejandro Cuadron, Kyle Chu, Shu Liu, Mark Zhao, Stephan Krusche, Alfons Kemper, Matei Zaharia, and Joseph E. Gonzalez. vCache: Verified Semantic Prompt Caching, 2025. URL https://arxiv.org/abs/2502.03771.

[23] Shanghao Shi, Xiao Wang, Chaoyu Zhang, Hao Li, Wenjing Lou, Thomas Hou, Yevgeniy Vorobeychik, Chongjie Zhang, and Ning Zhang. Think Twice Before You Act: Protecting LLM Agents Against Tool Description Poisoning via Isolated Planning, 2026. URL https://arxiv.org/abs/2606.20922.

[24] Zhengliang Shi, Yuhan Wang, Lingyong Yan, Pengjie Ren, Shuaiqiang Wang, Dawei Yin, and Zhaochun Ren. Retrieval Models Aren't Tool-Savvy: Benchmarking Tool Retrieval for Large Language Models, 2025. URL https://arxiv.org/abs/2503.01763.

[25] Nolan Smyth, Yorguin-Jose Mantilla-Ramos, Pascal Jr Tikeng Notsawo, Saskia Helbling, Alberto Tosato, Mohamed Amine Merzouk, Nouha Dziri, Gauthier Gidel, and Tommaso Tosato. Quantifying Overclaiming Propensity in Frontier LLM Agents, 2026. URL https://arxiv.org/abs/2609.20812.

[26] William R. Thompson. On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples. *Biometrika*, 25(3/4):285–294, 1933.

[27] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. MCPTox: A Benchmark for Tool Poisoning Attack on Real-World MCP Servers, 2025. URL https://arxiv.org/abs/2508.14925.

[28] Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, et al. Evo-Memory: Benchmarking LLM Agent Test-time Learning with Self-Evolving Memory, 2025. URL https://arxiv.org/abs/2511.20857.

[29] Jacob Whitehill, Paul Ruvolo, Tingfan Wu, Jacob Bergsma, and Javier Movellan. Whose Vote Should Count More: Optimal Integration of Labels from Labelers of Unknown Expertise. In *Advances in Neural Information Processing Systems*, 2009.

[30] Yuhang Wu, Xiangqing Shen, Fanfan Wang, Cangqi Zhou, Zhen Wu, Xinyu Dai, and Rui Xia. Optimizing RAG Rerankers with LLM Feedback via Reinforcement Learning, 2026. URL https://arxiv.org/abs/2604.02091.

[31] Hoyeol Yang, Woojung Song, Taewon Kim, Jonghyun Song, Seoyeon Park, and Yohan Jo. Agents' Overreliance on Unreliable Tools, 2026. URL https://arxiv.org/abs/2609.05587.

[32] Yi Yang, Zhennan Chen, Yihong Zhuang, Tiehan Fan, Yinan Chen, Jian Li, Jian Yang, and Ying Tai. RoMeRL: Balancing Feedback Coverage and the Memory-Reward Trap in Self-Evolving Agent Memory via Reduced-Order Utility States, 2026. URL https://arxiv.org/abs/2608.02508.

[33] Hengkai Ye, Zhechang Zhang, Jinyuan Jia, and Hong Hu. TRUSTDESC: Preventing Tool Poisoning in LLM Applications via Trusted Description Generation, 2026. URL https://arxiv.org/abs/2604.07536.

[34] Cesare Zavattari, Alessandro Tommasi, and Giuseppe Prencipe. One Human, N Agents: Audit-Budget Allocation for LLM Agent Fleets under Miscalibrated, Correlated Confidence, 2026. URL https://arxiv.org/abs/2607.28317.

[35] Chubin Zhang, Zhenglin Wan, Xingrui Yu, Pengfei Zhou, Wangbo Zhao, Jingxuan Wu, Yaxin Zhou, and Ivor Tsang. Don't Blindly Trust It: How Unreliable Feedback Breaks Tool-Using LLM Agents, 2026. URL https://arxiv.org/abs/2606.21409.

[36] Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, et al. MemRL: Self-Evolving Agents via Runtime Reinforcement Learning on Episodic Memory, 2026. URL https://arxiv.org/abs/2601.03192.

[37] Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, et al. Memento: Fine-tuning LLM Agents without Fine-tuning LLMs, 2025. URL https://arxiv.org/abs/2508.16153.

A Hyperparameters

Setting	Value
Score weights (w_s, w_c, w_p, w_e)	$(0.20, 0.40, 0.10, 0.30)$; library default $w_e = 0.10$
Context shrinkage κ	4
Fixed discount	report capped at 0.75; weights 0.45 report, 0.15 schema/judge
Learned trust	prior centred on uninformative, strength 2; per-tool shrinkage 4
Log-likelihood clip	± 4
Long-term counter update	1/4 of the short-term update
Change test	verified 0/1 outcomes, window $w = 8$, two-sided $\alpha = 0.01$
Time decay, ϵ-greedy	none
MemRL-style	$\lambda = 0.5$, learning rate 0.3, $Q_0 = 0.5$
Robust per-context memory	move on after 2 failures in a row (3 if unverified)
Seeds	30 (42–71), paired across policies
Gates M, W	30 shops (6 held out), 4 pages each, 4 epochs, 8 extractors
Gate M faults	20% timeouts on the flaky server, 2% transient failures on every tool
Gate M defection	`css_woo` wrong but reporting success after half the seen steps
Spot-check rate	20% of calls verified
Gate T	candidate pool 8, 20% of requests held out
Gates G, F	candidate pool 5
LLM, Gate M	Gemini 2.5 Flash, temperature 0, thinking off
LLM, Gate G	Gemini 2.5 Flash, temperature 0.7, thinking off, 2 samples per cell
LLM, Gate F	Gemini 2.5 Flash, temperature 0, default thinking

B Learned trust against the fixed discount

Gate	Scenario	Learned − fixed	Global-only − per-tool
M	Defection + spot	**−3.2** [−5.0, −1.4]	**−5.9** [−7.1, −4.8]
M	Spot check	**−3.2** [−5.2, −1.2]	**−8.3** [−9.6, −7.1]
M	Update + spot	**−3.1** [−5.2, −1.1]	**−10.0** [−11.9, −8.1]
M clean	Defection + spot	+0.4 [−1.2, +2.0]	**−2.7** [−3.9, −1.6]
M clean	Spot check	+0.4 [−0.9, +1.7]	−0.1 [−0.7, +0.5]
M clean	Update + spot	+0.1 [−1.0, +1.2]	+0.4 [−0.3, +1.1]
W	Redesign + spot	**−2.2** [−4.1, −0.3]	**−9.8** [−11.1, −8.5]
W	Spot check	**−2.0** [−3.8, −0.1]	**−9.0** [−10.1, −7.8]
T ToolBench	Drift + fooled	**+0.6** [+0.1, +1.1]	+0.1 [−0.2, +0.3]
T ToolBench	Drift + syco.	**+0.7** [+0.1, +1.2]	−0.0 [−0.3, +0.2]
T ToolBench	Fooled	**+0.5** [+0.0, +0.9]	−0.1 [−0.3, +0.2]
T ToolBench	Sparse verifier	**+2.9** [+2.4, +3.4]	+0.0 [−0.1, +0.1]
T ToolBench	Sycophantic	+0.5 [−0.1, +1.0]	+0.1 [−0.1, +0.3]
T ToolLens	Drift + fooled	−0.8 [−1.7, +0.1]	+0.1 [−0.4, +0.6]
T ToolLens	Drift + syco.	−0.5 [−1.2, +0.3]	−0.3 [−0.6, +0.0]
T ToolLens	Fooled	−0.7 [−1.7, +0.2]	**+0.5** [+0.0, +0.9]
T ToolLens	Sparse verifier	**+1.6** [+0.9, +2.3]	+0.0 [−0.3, +0.4]
T ToolLens	Sycophantic	**−1.2** [−2.2, −0.2]	**+0.5** [+0.1, +1.0]
G	Drift + fooled	+0.8 [−0.9, +2.5]	+0.2 [−0.8, +1.1]
G	Drift + syco.	+0.1 [−1.1, +1.3]	+0.2 [−0.3, +0.6]
G	Fooled	−0.1 [−0.8, +0.5]	+0.1 [−0.2, +0.4]
G	Sparse verifier	+0.4 [−0.0, +0.7]	+0.0 [−0.0, +0.1]
G	Sycophantic	+0.4 [−0.2, +1.0]	+0.1 [−0.1, +0.2]

Table 10: Learned source trust against the fixed discount (RRL full), and global-only against per-tool trust (RRL global), in every scenario with unverified feedback. Mean paired difference in overall success (points) over 30 seeds with 95% CI; bold where the CI excludes zero.

C Staged defection without the poisoned server

Overall / held-out success (%)	Defection	Defection + spot
Static reranker	38.3 / 65.7	38.3 / 65.7
LLM picks from tool list	40.0 / 61.8	40.0 / 61.8
Per-context memory	66.7 / 64.4	63.0 / 64.4
Per-context memory, robust	**68.2** / 65.3	**63.7** / 65.3
MemRL-style	33.8 / 58.5	17.2 / 4.0
RRL (global)	39.8 / 61.7	37.6 / 60.6
RRL full	52.0 / 62.2	42.9 / 63.3
RRL full, fixed rule	52.0 / 62.2	42.5 / 59.7
Best possible	83.6 / 82.2	83.6 / 82.2

Table 11: Staged defection on Gate M without the poisoned server.